

Local Randomized Neural Networks with Discontinuous Galerkin Methods for KdV-type and Burgers Equations

Summary of Sun and Wang (2024)

1 Introduction

The Korteweg-de Vries (KdV) equation and Burgers equation are fundamental nonlinear partial differential equations with significant applications in fluid dynamics, physics, and engineering. The KdV equation, characterized by its nonlinear nature and third-order derivative terms, is essential for studying shallow water wave behavior. Similarly, the Burgers equation, which combines nonlinear advection with diffusion, plays a crucial role in turbulence modeling and shock wave analysis.

Traditional numerical methods for solving these equations, such as finite difference, finite element, and discontinuous Galerkin approaches, often suffer from computational inefficiencies when dealing with complex wave phenomena. More recently, neural network-based methods have shown promise in solving PDEs due to their powerful approximation capabilities. However, these approaches typically rely on optimization solvers for training, which can lead to difficulties achieving high accuracy and efficiency due to the challenges of solving nonconvex optimization problems.

To address these limitations, this paper extends the Local Randomized Neural Network with Discontinuous Galerkin (LRNN-DG) methods to solve nonlinear KdV-type and Burgers equations using a space-time approach. Additionally, it introduces adaptive domain decomposition and a characteristic direction approach to enhance computational efficiency.

2 Neural Network Structure and Framework

2.1 Randomized Neural Networks

The LRNN-DG method employs randomized neural networks (RNNs), which differ from conventional neural networks in their training process. In RNNs, the weights between hidden layers are randomly assigned and kept fixed during training, while only the output layer parameters are determined using

a least-squares approach. This strategy significantly reduces the computational complexity associated with training.

The fully connected neural network structure is defined as:

$$N^{(1)}(s) = \rho(W^{(1)}s + b^{(1)}) \quad (1)$$

$$N^{(i)}(s) = \rho(W^{(i)}N^{(i-1)} + b^{(i)}), \quad i = 2, \dots, L \quad (2)$$

$$U(s) = W^{(L+1)}N^{(L)} \quad (3)$$

where $N^{(i)}$ denotes the i -th hidden layer with weight matrix $W^{(i)}$ and bias vector $b^{(i)}$, ρ is the activation function (Tanh or ReLU), L is the number of hidden layers, and $W^{(L+1)}$ is the weight matrix of the output layer.

The function space for RNNs can be written as:

$$M(\sigma) = \left\{ U(\alpha, \theta, t, x) = \sum_{j=1}^M \alpha_j^\sigma \phi^\sigma(j, t, x) : \begin{pmatrix} t \\ x \end{pmatrix} \in \sigma \right\} \quad (4)$$

where $\sigma \subset I \times \Omega$, $\phi^\sigma(j, t, x)$ are outputs of the last hidden layer with fixed parameters θ_j , $\{\alpha_j^\sigma\}_{j=1}^M$ are the trainable weights of the output layer, and M is the number of neurons in the last hidden layer.

2.2 Space-Time Domain Decomposition

To handle complex problems effectively, the LRNN-DG approach decomposes the computational domain into multiple subdomains, with separate RNNs assigned to each subdomain. The spatial domain Ω is decomposed into a mesh $\{T_h\}$ with N_s elements, and the temporal interval I is divided into N_t sub-intervals $D_\tau = \{I_i = (t_{i-1}, t_i), 0 = t_0 < t_1 < \dots < t_{N_t} = T\}$.

For a one-dimensional spatial domain $\Omega = (x_0, x_{N_s})$, T_h consists of elements $K_i = (x_{i-1}, x_i)$ for $i = 1, 2, \dots, N_s$. The space-time decomposition $M_h^\tau = D_\tau \times T_h$ comprises a total of $N_e = N_t N_s$ sub-elements.

The local randomized neural network function spaces are defined as:

$$V_h^\tau = \{v_h^\tau \in L^2(\Sigma) : v_h^\tau|_\sigma \in M_{RNN}(\sigma) \quad \forall \sigma \in D_\tau \times T_h\} \quad (5)$$

$$Q_h^\tau = \{q_h^\tau \in [L^2(\Sigma)]^d : q_h^\tau|_\sigma \in [M_{RNN}(\sigma)]^d \quad \forall \sigma \in D_\tau \times T_h\} \quad (6)$$

3 LRNN-DG Methods for Nonlinear PDEs

3.1 LRNN-DG Method for KdV Equations

The paper develops two approaches for solving the KdV equation: the LRNN-DG and LRNN-C¹DG methods.

3.1.1 Problem Formulation

Consider the KdV equation with nonlinear term:

$$u_t(t, x) + b_{KdV}(u, \frac{\partial u}{\partial x}) + u_{xxx}(t, x) = 0, \quad (t, x) \in I \times \Omega \quad (7)$$

Subject to boundary conditions:

$$u(t, x_0) = g_0 \quad (8)$$

$$\frac{\partial u}{\partial x}(t, x_{N_s}) = g_1 \quad (9)$$

$$\frac{\partial^2 u}{\partial x^2}(t, x_{N_s}) = g_2 \quad (10)$$

And initial condition:

$$u(t_0, x) = u_0 \quad (11)$$

3.1.2 LRNN-DG Formulation

The LRNN-DG scheme for solving the nonlinear KdV equation is: Find $u_h^\tau \in V_h^\tau$ such that

$$B_{KdV}(u_h^{\tau(n+1)}, v_h^\tau) + (b_{KdV}^L(u_h^{\tau(n+1)}, u_h^{\tau(n)}), v_h^\tau) = l_{KdV}(v_h^\tau) + (b_{KdV}^R(u_h^{\tau(n)}), v_h^\tau) \quad \forall v_h^\tau \in V_h^\tau \quad (12)$$

for $n = 0, 1, \dots$

Here, B_{KdV} represents the bilinear form for the linear part:

$$\begin{aligned} B_{KdV}(u_h^\tau, v_h^\tau) &= \int_{\Sigma} \frac{\partial^\tau u_h^\tau}{\partial^\tau t} v_h^\tau dt dx - \sum_{i=0}^{N_t-1} \int_{T_h} [u_h^\tau(t_i, x)] \cdot \{v_h^\tau(t_i, x)\} dx \\ &\quad - \sum_{i=1}^{N_t-1} \int_{T_h} \eta_1 [u_h^\tau(t_i, x)] \cdot [v_h^\tau(t_i, x)] dx + \int_{\Sigma} \frac{\partial^h u_h^\tau}{\partial^h x} \frac{\partial^2 v_h^\tau}{\partial^h x^2} dt dx + \dots \end{aligned} \quad (13)$$

The terms b_{KdV}^L and b_{KdV}^R denote linearizations of the nonlinear term computed using Newton or Picard methods. For example, for the nonlinear term uu_x , the Newton linearization gives:

$$b_{KdV}^L(u_h^{\tau(n+1)}, u_h^{\tau(n)}) = \frac{\partial u_h^{\tau(n+1)}}{\partial x} u_h^{\tau(n)} + u_h^{\tau(n+1)} \frac{\partial u_h^{\tau(n)}}{\partial x} \quad (15)$$

$$b_{KdV}^R(u_h^{\tau(n)}) = \frac{\partial u_h^{\tau(n)}}{\partial x} u_h^{\tau(n)} \quad (16)$$

3.1.3 LRNN-C¹DG Formulation

The LRNN-C¹DG method enforces initial conditions, boundary conditions, and continuity conditions at selected collocation points to integrate local neural networks across different subdomains.

The weak form is:

$$B_{KdV}^\sigma(u_h^{\tau(n+1)}, v_h^\tau) + \left(b_{KdV}^L(u_h^{\tau(n+1)}, u_h^{\tau(n)}), v_h^\tau \right)_\sigma = \left(b_{KdV}^R(u_h^{\tau(n)}), v_h^\tau \right)_\sigma \quad \forall v_h^\tau \in V_h^\tau \quad (17)$$

Additionally, the following conditions are imposed at collocation points:

$$u_h^\tau(t_\tau^0, x_\tau^0) = u_0 \quad \forall (t_\tau^0, x_\tau^0) \in P_\tau^0 \quad (\text{Initial condition}) \quad (18)$$

$$[u_h^\tau(t_\tau^i, x_\tau^i)] = 0 \quad \forall (t_\tau^i, x_\tau^i) \in P_\tau^i \quad (\text{Temporal edge continuity}) \quad (19)$$

$$u_h^\tau(t_h^-, x_h^-) = g_0 \quad \forall (t_h^-, x_h^-) \in P_h^- \quad (\text{Left boundary condition}) \quad (20)$$

$$\frac{\partial u_h^\tau}{\partial x}(t_h^+, x_h^+) = g_1, \quad \frac{\partial^2 u_h^\tau}{\partial x^2}(t_h^+, x_h^+) = g_2 \quad \forall (t_h^+, x_h^+) \in P_h^+ \quad (\text{Right boundary conditions}) \quad (21)$$

$$[u_h^\tau(t_h^i, x_h^i)] = 0, \quad \left[\frac{\partial u_h^\tau}{\partial x}(t_h^i, x_h^i) \right], \quad \left[\frac{\partial^2 u_h^\tau}{\partial x^2}(t_h^i, x_h^i) \right] = 0 \quad \forall (t_h^i, x_h^i) \in P_h^i \quad (\text{Spatial edge continuity}) \quad (22)$$

3.2 LRNN-DG Method for Burgers Equation

The paper also extends the LRNN-DG framework to solve the Burgers equation.

3.2.1 Problem Formulation

Consider the Burgers equation with Dirichlet boundary conditions:

$$u_t + u(\nabla u \cdot I) - \epsilon \Delta u = 0 \quad \text{in } I \times \Omega \quad (23)$$

$$u(t_0, x) = u_0 \quad \text{on } \{t_0\} \times \Omega \quad (24)$$

$$u(t, x) = g \quad \text{on } I \times \partial\Omega \quad (25)$$

where $I = (t_0, T)$ and $\Omega \subset \mathbb{R}^d$ represent the time and space domains, and I is a d -dimensional vector $[1, 1, \dots, 1]^T$.

3.2.2 LRNN-DG Formulation for Burgers Equation

The LRNN-DG method for the Burgers equation is: Find $u_h^\tau \in V_h^\tau$ such that

$$B_{Bur}(u_h^{\tau(n+1)}, v_h^\tau) + \left(b_{Bur}^L(u_h^{\tau(n+1)}, u_h^{\tau(n)}), v_h^\tau \right) = l_{Bur}(v_h^\tau) + \left(b_{Bur}^R(u_h^{\tau(n)}), v_h^\tau \right) \quad \forall v_h^\tau \in V_h^\tau \quad (26)$$

where the bilinear form B_{Bur} is:

$$B_{Bur}(u_h^\tau, v_h^\tau) = \int_{\Sigma} \frac{\partial^\tau u_h^\tau}{\partial^\tau t} v_h^\tau dt dx + \int_{\Sigma} \nabla_h u_h^\tau \cdot \nabla_h v_h^\tau dt dx \quad (27)$$

$$- \sum_{i=0}^{N_t-1} \int_{T_h} [u_h^\tau(t_i, x)] \cdot \{v_h^\tau(t_i, x)\} dx - \sum_{i=1}^{N_t-1} \int_{T_h} \eta[u_h^\tau(t_i, x)] \cdot [v_h^\tau(t_i, x)] dx \quad (28)$$

$$- \int_{D_\tau \times E_h} (\llbracket u_h^\tau \rrbracket \cdot \{\nabla_h v_h^\tau\} + \llbracket v_h^\tau \rrbracket \cdot \{\nabla_h u_h^\tau\} - \eta[u_h^\tau] \cdot \llbracket v_h^\tau \rrbracket) dt ds \quad (29)$$

3.2.3 LRNN-C¹DG Formulation for Burgers Equation

Similar to the KdV equation, the LRNN-C¹DG method for the Burgers equation enforces boundary and continuity conditions at collocation points:

$$u_h^\tau(t_\tau^0, x_\tau^0) = u_0 \quad \forall (t_\tau^0, x_\tau^0) \in P_\tau^0 \quad (30)$$

$$[u_h^\tau(t_\tau^i, x_\tau^i)] = 0 \quad \forall (t_\tau^i, x_\tau^i) \in P_\tau^i \quad (31)$$

$$u_h^\tau(t_h^\partial, x_h^\partial) = g \quad \forall (t_h^\partial, x_h^\partial) \in P_h^\partial \quad (32)$$

$$[u_h^\tau(t_h^i, x_h^i)] = 0, \quad [\nabla u_h^\tau(t_h^i, x_h^i)] = 0 \quad \forall (t_h^i, x_h^i) \in P_h^i \quad (33)$$

4 Adaptive Domain Decomposition and Characteristic Direction Approach

4.1 Adaptive Domain Decomposition

To enhance accuracy and efficiency, the paper introduces adaptive domain decomposition, where the domain decomposition is guided by error estimators that reflect the error distribution:

1. Start with a uniform rectangular mesh $M_{\tau h}^{(0)}$
2. Calculate interior residuals $R_{\sigma_i} = Lu - f$ for each subdomain σ_i
3. Compute local error estimators $\{R_i^h = (h_{\sigma_i}^2 \|R_{\sigma_i}\|_{\sigma_i})^{1/2}, i = 1, 2, \dots, N_e\}$
4. Calculate the sum of residuals $R_{tot} = \sum_{i=1}^{N_e} (R_i^h)^2$
5. Rearrange local residuals from largest to smallest
6. Introduce a parameter $\beta > 0$ and determine the smallest N_r such that $\sum_{i=1}^{N_r} (R_i^h)^2 \geq \beta R_{tot}$
7. Refine the first N_r subdomains with the largest local error estimator, leading to a new decomposition $M_{\tau h}^{(1)}$
8. Repeat until desired accuracy is achieved or maximum iterations reached

4.2 Characteristic Direction Approach

If the characteristic direction of the solution is known in advance (e.g., through traveling wave analysis), the LRNN-DG method can be made more effective by:

1. Deriving a priori information about the solution's shape from known initial conditions
2. Partitioning the domain along the characteristic direction based on this prior information
3. Introducing distinct local networks to approximate the solution in each subdomain

Additionally, the method can incorporate wavelet basis functions aligned with the characteristic directional information to enhance the network's approximation capability, achieving highly accurate numerical results efficiently.

5 Numerical Examples and Results

5.1 Generalized KdV Equation with Soliton Solution

The first example tests the LRNN-DG methods on the generalized KdV equation with a small coefficient for the third derivative term:

$$u_t + u_x + u^3 u_x + \epsilon u_{xxx} = 0, \quad (t, x) \in I \times \Omega \quad (34)$$

where $\Omega = [-2, 3]$, $I = [0, 2]$, and $\epsilon = 0.2058 \times 10^{-4}$. The exact solution is a soliton:

$$u(t, x) = A \operatorname{sech}^2(K(x - x_0) - \omega t) \quad (35)$$

where $A = 0.2275$, $K = 3(A^3/40\epsilon)^{1/2}$, $\omega = K(1 + A^3/10)$, and $x_0 = 0.5$. Results showed:

- Both LRNN-DG and LRNN-C¹DG methods achieved high accuracy on uniform meshes
- With 320 DoF per subdomain and mesh size $h = \tau = 1/4$, the L^2 errors reached 2.14×10^{-6} and 1.11×10^{-6} respectively
- The adaptive mesh approach significantly improved accuracy, with L^2 error reaching 9.51×10^{-7} using 148 subdomains
- The characteristic mesh approach (using only 10 subdomains) provided remarkable efficiency, achieving L^2 errors as low as 4.69×10^{-5} with 320 DoF per subdomain
- Using wavelet activation functions aligned with the characteristic direction yielded even better results, with L^2 errors of 3.90×10^{-6} using just 80 DoF per subdomain

5.2 KdV Equation with Double Solitons Collision

The second example examines the KdV equation modeling the collision of two solitons:

$$u_t + uu_x + \epsilon u_{xxx} = 0, \quad (t, x) \in I \times \Omega \quad (36)$$

with initial condition:

$$u_0(x) = 3c_1 \text{sech}^2(k_1(x - x_1)) + 3c_2 \text{sech}^2(k_2(x - x_2)) \quad (37)$$

where $c_1 = 0.3$, $c_2 = 0.1$, $x_1 = 0.4$, $x_2 = 0.8$, and $k_i = \frac{1}{2}\sqrt{\frac{c_i}{\epsilon}}$ for $i = 1, 2$, with $\epsilon = 4.84 \times 10^{-4}$.

Results showed:

- On uniform meshes, even with fine refinement (81 elements), the LRNN-C¹DG method struggled to capture some of the finer details of the soliton collision
- The adaptive mesh approach significantly improved solution quality, capturing the collision dynamics effectively with just 43-58 elements

5.3 2D Burgers Equation

The third example investigated the 2D Burgers equation:

$$u_t + u(u_x + u_y) - \epsilon \Delta u = 0, \quad (t, x, y) \in I \times \Omega \quad (38)$$

with exact solution $u = 1/(1 + e^{(x+y-t)/(2\epsilon)})$.

Results demonstrated:

- For $\epsilon = 0.1$, both LRNN-DG and LRNN-C¹DG methods achieved high accuracy, with the LRNN-DG method outperforming the LRNN-C¹DG method
- With 320 DoF per subdomain and mesh size $h = \tau = 1/4$, L² errors reached 4.22×10^{-7} and 1.03×10^{-6} respectively
- For the more challenging case of $\epsilon = 0.01$, the LRNN-DG method on a uniform mesh achieved L² errors of 4.19×10^{-3} with 320 DoF and $h = \tau = 1/6$
- The adaptive mesh approach significantly improved efficiency for $\epsilon = 0.01$, with L² error reaching 5.58×10^{-3} using 148 elements and 160 DoF per subdomain

6 Implications and Advantages

6.1 Computational Efficiency

- **Reduced training complexity:** By fixing the hidden layer parameters randomly and only training the output layer weights through least-squares methods, LRNN-DG methods avoid the computational burden and convergence issues associated with nonconvex optimization.

- **Space-time approach:** The methods solve time-dependent problems in a single step without time-stepping, significantly reducing computational cost for long-time simulations and avoiding error accumulation over time iterations.
- **Adaptive mesh refinement:** The adaptive domain decomposition strategy focuses computational resources on regions with high error, achieving higher accuracy with fewer elements.
- **Characteristic mesh advantage:** By aligning mesh structures with the characteristic direction of the solution, the approach achieves remarkable accuracy with minimal subdomains (e.g., using only 10 subdomains for the KdV soliton problem).

6.2 Approximation Power and Accuracy

- **High accuracy with fewer DoF:** LRNN-DG methods achieve high accuracy with relatively few degrees of freedom compared to traditional methods.
- **Adaptive basis functions:** The incorporation of problem-specific information through wavelet activation functions aligned with characteristic directions significantly enhances the network’s approximation capability.
- **Handling nonlinear phenomena:** The methods effectively capture complex nonlinear wave phenomena such as solitons and their interactions.
- **Performance on challenging problems:** The methods demonstrate strong performance on problems with small diffusion coefficients (e.g., Burgers equation with $\epsilon = 0.01$), which are challenging for many numerical approaches.

6.3 Flexibility and Extensions

- **Framework extensibility:** The LRNN-DG framework can be applied to various nonlinear PDEs by adapting the weak formulation and linearization strategies.
- **Multiple integration options:** The approach offers different integration strategies (LRNN-DG and LRNN-C¹DG) to accommodate different problem requirements.
- **Dimension independence:** The method extends naturally to multi-dimensional problems, as demonstrated by the 2D Burgers equation example.

- **Prior information incorporation:** The framework allows for integration of a priori knowledge about the solution, significantly enhancing performance for problems where such information is available.

6.4 Challenges and Future Directions

- **Theoretical foundation:** A rigorous numerical analysis of LRNN-DG methods for nonlinear problems is needed to establish theoretical convergence rates and stability guarantees.
- **Error estimation:** Developing more reliable and efficient error estimators to guide adaptive refinement remains an important research direction.
- **Optimal activation functions:** The selection of activation functions that best suit specific problem types needs further investigation.
- **Automation of characteristic mesh generation:** Developing systematic approaches to identify and exploit characteristic information for more general problems would enhance the method's applicability.